

using the *subprocess* Python module. We check the file descriptors *process.stdout* (standard output of the sub-process) and if that does not return any data, then *process.stderr* (the standard error of the sub-process). We then send the data returned from the sub-process to the sender of the message.

Like before, from Pidgin (logged in as a different user), try sending commands like *ls*, *cat /proc/cpuinfo*, etc, to this bot's GTalk ID. You should see the expected results returned as messages in your Pidgin window.

This bot has no access-control—any user in its buddy list could send commands and have them executed. Let's add a simple check for the message sender's ID, so we can restrict command execution permission to a single buddy. Any other users sending commands to the bot should receive an "Access denied" type of error message. Replace the *message_handler()* function with this new one:

```
def message_handler(connect_object,message_node):

    admin = "admin_user@gmail.com"
    from_user = message_node.getFrom().getStripped()

    if admin == from_user: # allow to execute command only if admin
        requested

        command = str(message_node.getBody())

        process = subprocess.Popen(command,shell=True,stdout=subprocess.PIPE, stderr=subprocess.PIPE)
        message = process.stdout.read()
        if message!="":
            message=process.stderr.read()

        else:
            message="Access denied\nContact system admin"

        connect_object.send( xmpp.Message( message_node.getFrom()
, message))
```

Find invisible users

Most instant messaging protocols support invisible users, so that anyone can remain online without being noticed by anyone in their buddy list. GTalk is no exception. When I started playing with XMPP, I noticed an interesting thing about GTalk's implementation: it implements user 'invisibility' on the client side, not the server side. We can easily find such invisible-yet-online users in our buddy list by listening to the presence notifications. (As explained before, whenever a client joins a server/network, it sends a presence notification for the user.) When a user authenticates to the server and becomes invisible, the client sends an 'unavailable' presence to all buddies in the roster. Thus, presence notifications with a presence type of 'unavailable' means that the user is in invisible mode. All GTalk clients ignore this type of presence; hence, those users are not shown in the buddy list. But we can write our own Python program to grab the

list of invisible users:

```
#!/usr/bin/python -W ignore:DeprecationWarning

import xmpp

users="user@gmail.com"
password="password"
servers="gmail.com"

def presenceHandler(conn, presence):
    if presence:
        if presence.getType() == "unavailable":
            print presence.getFrom().getStripped()

print "Invisible users:"

jid = xmpp.JID(user)
connection = xmpp.Client(server, debug={})
connection.connect()
result = connection.auth(jid.getNode(), password,"Client Name")

connection.RegisterHandler('presence',presenceHandler)
connection.sendInitPresence()

while connection.Process(1):
    pass
```

Writing a GUI front end for your client

So far, our simple Python GTalk client using the XMPP module was all about command-line programs that must be run in a terminal. You can develop a Graphical User Interface to make your client more user-friendly and increase usability. The two most popular GUI toolkits for use with Python are Qt and GTK. Qt is the base of the KDE desktop environment, while GTK is the base for the GNOME desktop environment. While writing the GUI, it is important to remember that you need to use threading to keep the XMPP and GUI part in separate threads. The *connection.Process(1)* function is to be called in an infinite loop. In the above programs, we used a *while* loop. In the case of Qt or GTK, both maintain a window by using an event loop—so using another infinite loop inside the Qt/ GTK window classes will, at some point, result in 'frozen' and unusable windows. Instead, use the Python threading module to run the XMPP part in another thread.

Hope you enjoyed the hacks around Google Talk and XMPP. Happy hacking till we meet again. **END** 🐧

By: Sarath Lakshman

The author is a Hacktivist of Free and Open Source Software from Kerala. He loves working on the GNU/Linux environment and contributes to the PTVi video editor project. He is also the developer of SLYNIX, a distro for newbies. He blogs at www.sarathlakshman.info